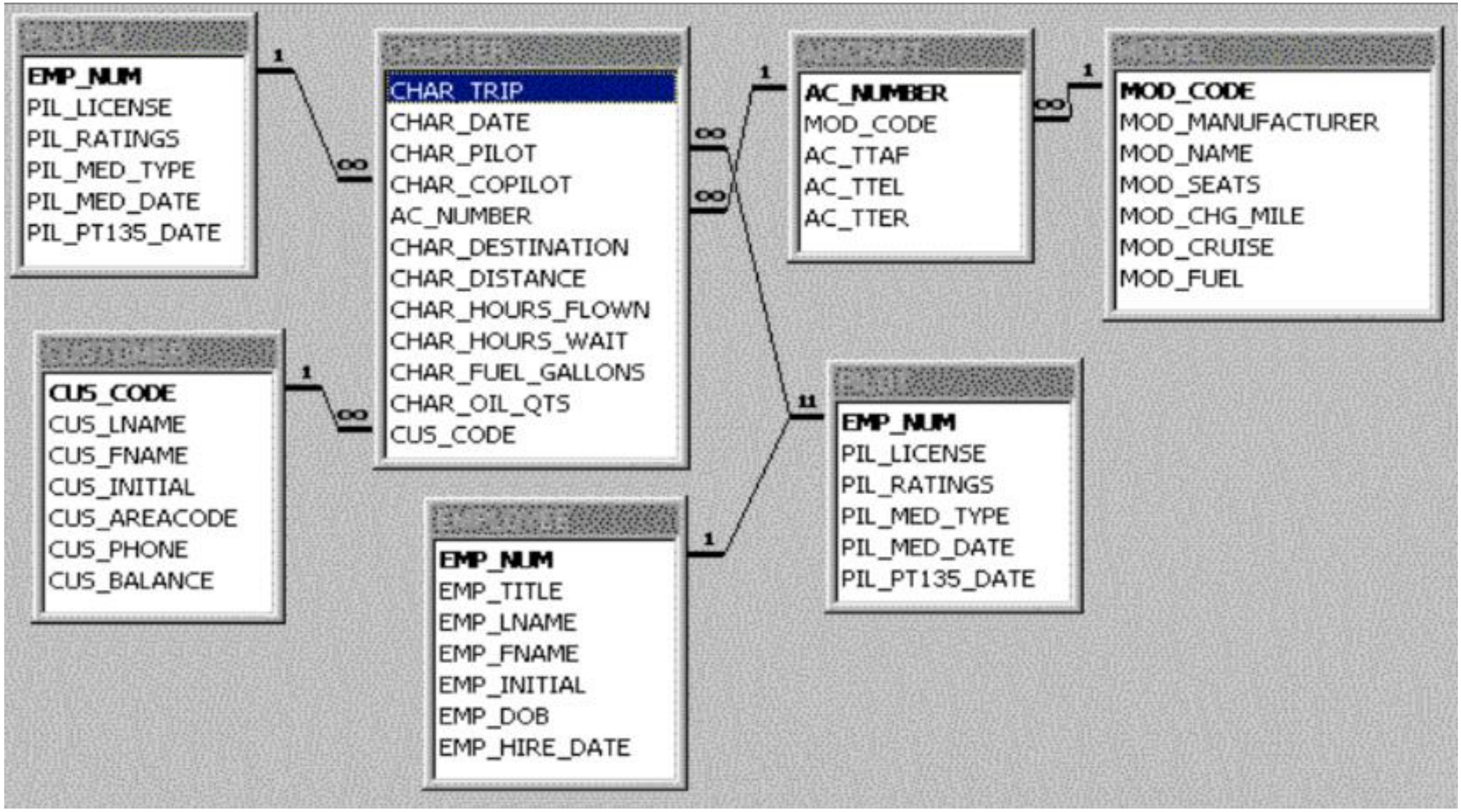
The Robcor Anomaly

Navigating the Data Warehousing Trap in Aviation Analytics



Gemini Notebook




```
SELECT * FROM DW.aircraft;
```

The Operational Brief

ROBCOR, Inc. is a rapidly growing on-demand **aviation charter** service requiring its first **decision support system**. The charter operations manager requires the ability to drill down by pilot, aeroplane model, and time periods.



Target 01: Revenue

- Total revenue each month/year (charter distance × model charge per mile).



Target 02: Hours

Total hours flown by each individual pilot.

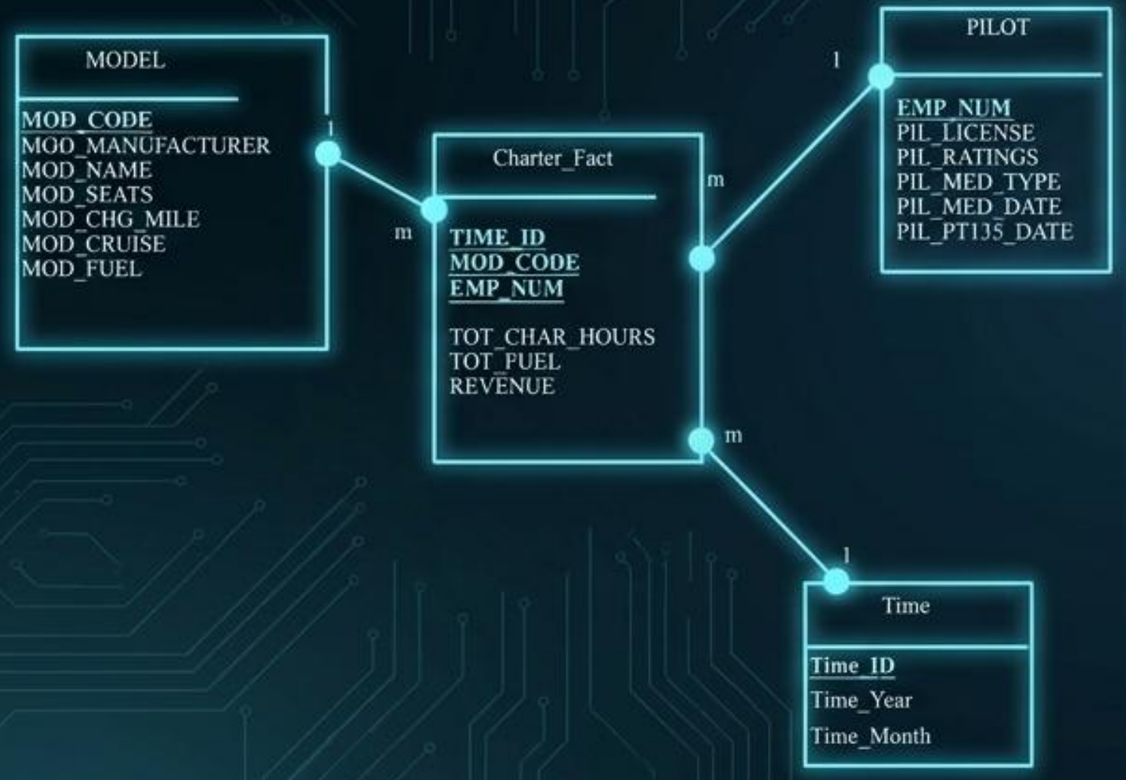


Target 03: Fuel

Total fuel consumed by each aeroplane model.

Gemini Notebook

The Initial Blueprint




```
create table charter_fact as
select C.Char_Pilot as EMP_Num,
      M.Mod_Code,
      to_char(C.Char_Date, 'YYYYMM') as Time_ID,
      sum(C.Char_Hours_Flown) as Tot_Char_Hours,
      sum(C.Char_Fuel_Gallons) as Tot_Fuel,
      sum(C.Char_Distance * M.Mod_chg_mile) as Revenue
from dw.Charter C, dw.Model M, dw.Aircraft A
```

Gemini Notebook



Rule #1: Never Trust the Operational Database

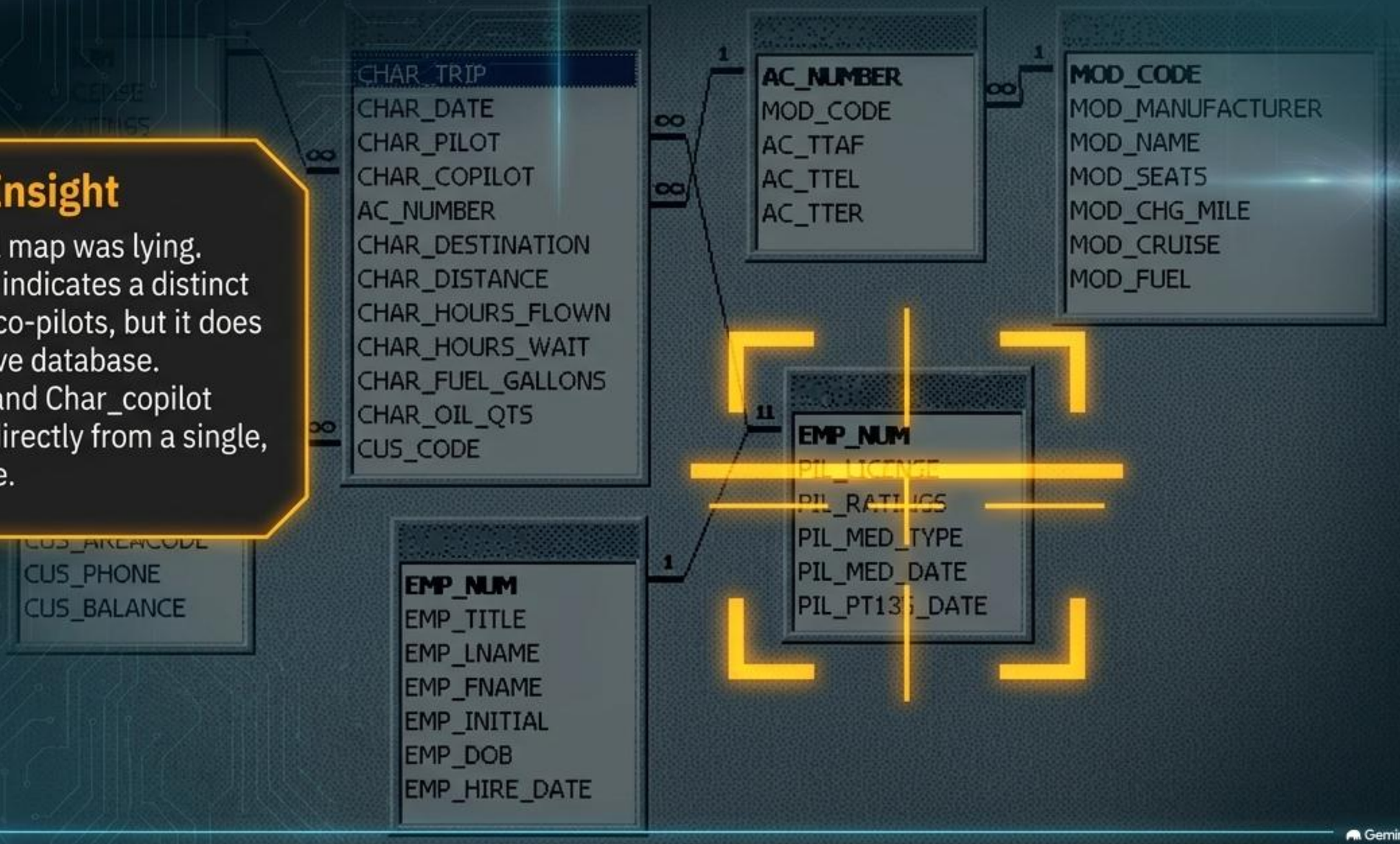
Data exploration is mandatory. We cannot determine if our warehouse is correct merely by reading the theoretical SQL blueprint. We must interrogate the raw operational inputs.

Gemini Notebook

Anomaly 01: The Phantom Table

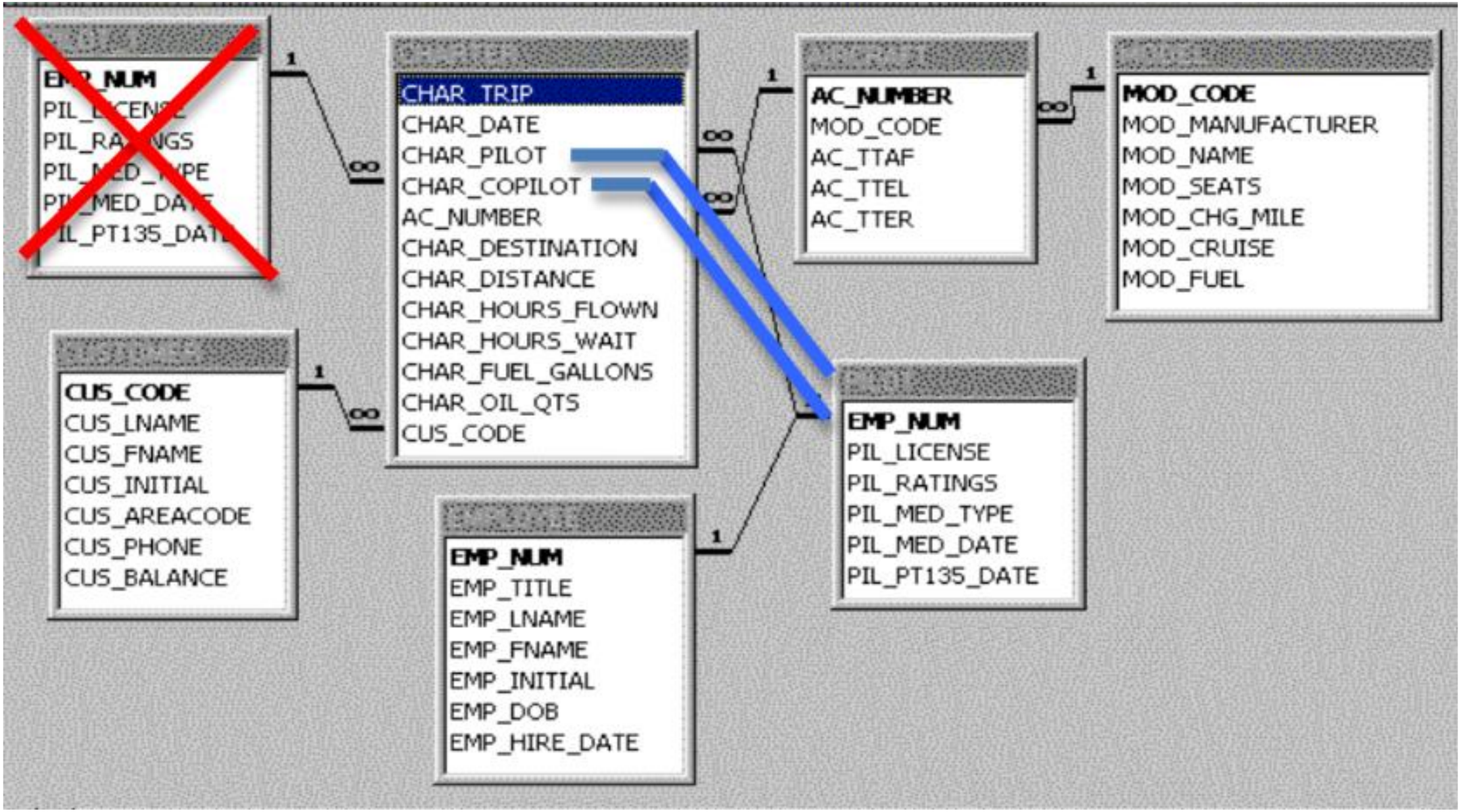
Diagnostic Insight

The architectural map was lying.
The E/R diagram indicates a distinct Pilot_1 table for co-pilots, but it does not exist in the live database.
Both Char_pilot and Char_copilot attributes draw directly from a single, unified Pilot table.



Gemini Notebook

Revised ERD


CHAR_TRIP
CHAR_DATE
CHAR_PILOT
CHAR_COPILOT
AC_NUMBER
CHAR_DESTINATION
CHAR_DISTANCE
CHAR_HOURS_FLOWN
CHAR_HOURS_WAIT
CHAR_FUEL_GALLONS
CHAR_OIL_QTS
CUS_CODE
AC_NUMBER
MOD_CODE
AC_TTAF
AC_TTEL
AC_TTER
MOD_CODE
MOD_MANUFACTURER
MOD_NAME
MOD_SEATS
MOD_CHG_MILE
MOD_CRUISE
MOD_FUEL
CUS_CODE
CUS_LNAME
CUS_FNAME
CUS_INITIAL
CUS_AREACODE
CUS_PHONE
CUS_BALANCE
EMP_NUM
EMP_TITLE
EMP_LNAME
EMP_FNAME
EMP_INITIAL
EMP_DOB
EMP_HIRE_DATE
EMP_NUM
PIL_LICENSE
PIL_RATINGS
PIL_MED_TYPE
PIL_MED_DATE
PIL_PT135_DATE
1
∞

Anomaly 02: The Operational Noise

```
SQL> select char_trip, count(*)
  2  from dw.charter
  3  group by char_trip
  4  having count(*) > 1;

 CHAR_TRIP  COUNT(*)
---------- ---------
     10268         2

SQL> select *
  2  from dw.charter
  3  where char_trip = 10268;

CHAR_TRIP   CHAR_DATE      CHAR_PILOT CHAR_COPILOT     AC_NU    CHA
---------- ---------- ---- ---------- ------------ --- -----    ---
CHAR_HOURS_FLOWN CHAR_HOURS_WAIT  CHAR_FUEL_GALLONS OEUAN CUS_CODE
---------- ----- --------- -----  ---- ------------ --- - --- -----
     10268  06/JUN/97                105              2778V BNA     387
       2.2                          3.6           86          1   10012
     10268  06/JUN/97                106          104 1484P MQY     302
       1.8                            0         60.4          2   10019

SQL>
```

VERIFIED

Database Interrogation Results:

Two entirely distinct charter records share the exact same CHAR_TRIP ID in the operational system.

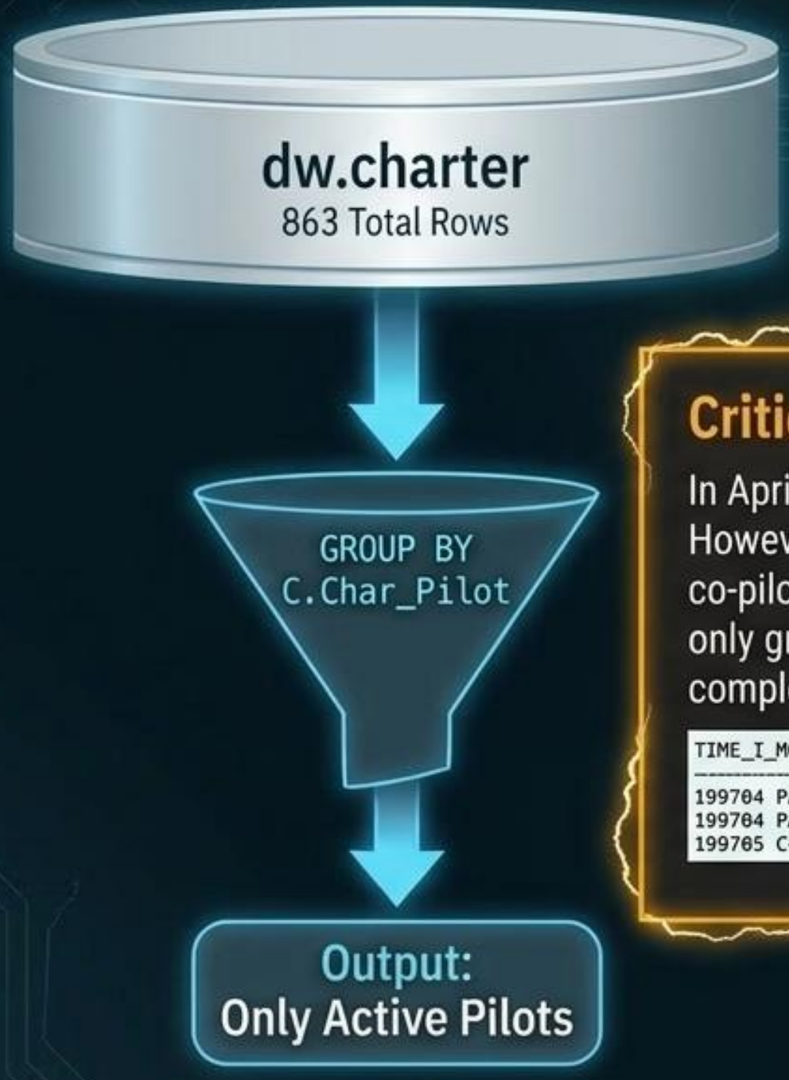
Analyst's Verdict:

The input data is undeniably unclean, but it is harmless noise. Because CHAR_TRIP is not utilized in our fact table aggregations, this operational error does not corrupt the data warehouse. We proceed.

Gemini Notebook

Anomaly 03: The Missing Co-Pilot

Should we consider copilot when calculating the fact table?



Critical Data Loss: Employee 101

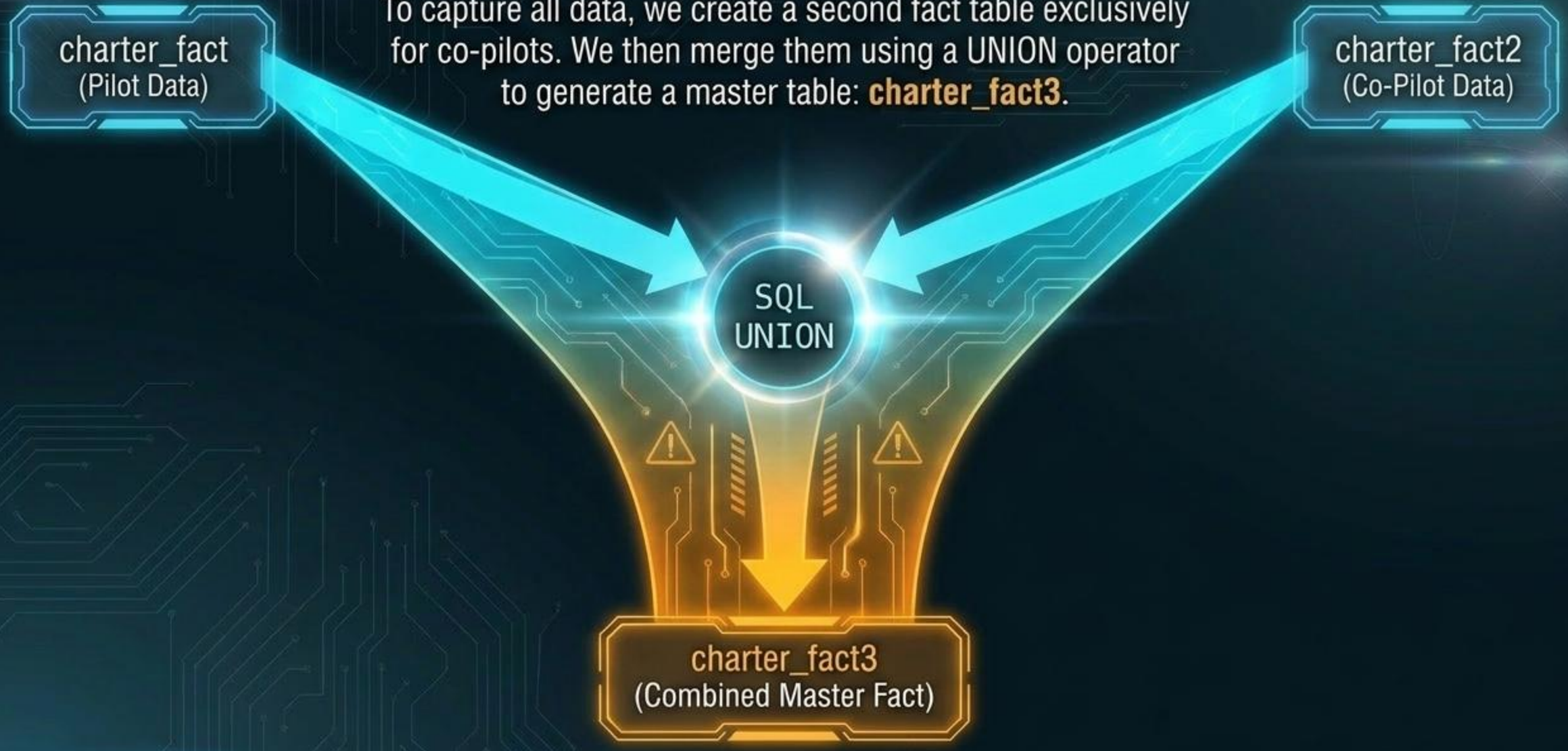
In April 1997, Employee 101 flew 3 trips as a pilot. However, Employee 101 also flew 1 trip as a co-pilot (5.6 hours). Because our initial schema only groups by Char_Pilot, these 5.6 hours have completely vanished from the analytics.

TIME_I_MOD_CODE	CHAR_PILOT	CHAR_HOURS_FLOWN	TOT_FUEL
199704 PA23-250	101	5.6	
199704 PA23-250	101	5.6	
199705 C-98A	101		

Gemini Notebook

The Obvious Fix: Merging Facts

To capture all data, we create a second fact table exclusively for co-pilots. We then merge them using a UNION operator to generate a master table: **charter_fact3**.



Gemini Notebook

The Double-Dip Trap

While the hours calculate perfectly, merging the two fact tables triggers a massive double-dip in business metrics. The fuel and revenue calculations are fundamentally broken.

Metric	Value	Status
Total Charter Hours	1053.2 hours	CORRECT
Total Fuel	50,534.9 gallons	⚠ AMBER ALERT: OVERCOUNT
Total Revenue	$447,900.57	⚠ AMBER ALERT: OVERCOUNT

Gemini Notebook

Anatomy of the Overlap

The Reality

1 Aeroplane

2 Personnel (Pilot A, Co-Pilot B)

100 gallons burned
$1,000 generated



⚠ The System Error ⚠

Pilot A logs: 100 gallons / $1,000

Co-Pilot B logs: 100 gallons / $1,000

+

Resulting Output: 200 gallons burned and $2,000 generated for a single physical flight.

Gemini Notebook

Diagnostic Framework: Measure Typology

Individual Measures

e.g., Flying Hours

Each employee accrues these independently. When they fly together, they both earn their own distinct hours.

✓ Safe to aggregate via UNION.

Company Measures

e.g., Total Fuel, Total Revenue

These are tied to the event (the trip), not the individual. The revenue is $1,000 regardless of whether there is one pilot or two.

✗ Unsafe to merge.
Causes overlap corruption.

Gemini Notebook

The Compromised Schema



The Attempt: We could create a combined fact table containing only employee numbers and hours, stripping out the problematic fuel and revenue data entirely.

The Verdict: Severely limited utility. A decision support system that cannot report on revenue or fuel consumption fails the core business brief. We must preserve all metrics without double-counting.

Gemini Notebook

The Final Architecture: Parallel Schemas

Pilot Fact Schema



Co-Pilot Fact Schema



Analyst Note: The safest and most accurate architecture is to maintain two distinct, parallel star star schemas. Because a single charter transaction contains two distinct personnel roles that overlap on company measures, combining them is structurally unviable.

Conclusion: We retain full drill-down capability for both pilots and co-pilots without corrupting the overarching company revenue and fuel metrics.

Gemini Notebook